Decision Analytics Journal 9 (2023) 100362

Contents lists available at ScienceDirect

Decision Analytics Journal

journal homepage: www.elsevier.com/locate/dajour

A community detection model using node embedding approach and graph convolutional network with clustering technique

Riju Bhattacharya [a,*], Naresh Kumar Nagwani [a], Sarsij Tripathi [b]

[a] Department of Computer Science & Engineering, National Institute of Technology, Raipur, India
[b] Department of Computer Science & Engineering, Motilal Nehru National Institute of Technology, Allahabad, India

ARTICLE INFO

Keywords:
Community identification
Graph convolutional network
Graph networks
Node embedding
Node2vec
Clustering

ABSTRACT

The communities in a network have distinct characteristics and interrelationships. Community discovery methods based on node embedding and deep learning have surpassed spectral clustering and statistical inference as the preferred methods for handling high-dimensional network data. Community detection in graph networks using graph neural networks (GNNs) is a rapidly expanding field of study that has attracted much attention recently. Inspired by the graph convolutional network (GCN) approach, promising results in evaluating graph structure data, a different method for discovering communities using node embedding using Node2Vec via graph convolutional networks (CD2NE-GCN) model has been proposed. It has been evaluated against the most recent standard methods of graph attention network (GAT), GCN-based approach for Unsupervised Community Detection (GUCD), and Graph Adversarial Networks (GAN). In this paper, a technique for community detection that combines the ideas of graph neural network design with node embedding via message forwarding has been presented. The experimentation has made use of five benchmark datasets: Karate, Cora, CiteSeer, Facebook, and DBLP. In the first set of trials, the node2vec uses the skip-gram with negative sampling (SGNS) approach via biased random walks of the second order to produce node embeddings and feed them to the graph convolutional network. The weighted average matrix and scaled normalized matrices of low-dimensional vertices have been calculated. In the next set of experiments, the average weighted matrix was sent to the 3-layer GCN with the activation function for generating feature vectors and applied clustering technique for predicting communities. The CD2NE-GCN approach outperforms previous literature by detecting graph network communities with accuracy scores of 98.45%, 99.04%, 96.39%, 98.79%, and 97.67% for Karate, Cora, CiteSeer, Facebook, and DBLP datasets, respectively. Thus, graph convolutional network models with a node embedding approach enhance the accuracy.

1. Introduction

Community detection is a crucial field of research in network analysis because it enables us to reveal the underlying structures of complex networks. A community in a network refers to a collection of nodes that are highly associated with each other, but have weak connections to the rest of the network. Finding these clusters of nodes helps us understand the modular structure of the network and locate related sets of nodes [1]. There are many different community detection algorithms, each with its own strengths and weaknesses. As a result of its importance, Community detection has been thoroughly explored by many different researchers, and many algorithms have been proposed to facilitate it. Several widely used traditional algorithms have been proposed, such as those using statistical modeling [2], modularity optimization [3], matrix factorization [4], and spectral clustering [5] have been employed for small and mid size networks. However, because of the significant computational and space costs associated with them, they are not suitable for use with large networks or networks that contain high-dimensional features. Because of the abundance of nonlinear structure information in real-world networks, conventional models have limited practical utility. Therefore, high-performance computation-based methods that are more robust are needed. However, because of the significant computational and space costs associated with them, they are not suitable for use with large networks or networks that contain high-dimensional features. Popular community discovery approach spectral clustering (SC) has been extensively addressed. An approach based on graph spectral characteristics and Laplacian matrix eigenvectors has been developed in 1973 [6]. The spectral clustering method has the advantage of not assuming a cluster shape. Lanczos [7] algorithms can efficiently implement the algorithm for big data sets. The spectral

* Corresponding author.
E-mail addresses: rbhattacharya.phd2018.cs@nitrr.ac.in (R. Bhattacharya), nknagwani.cs@nitrr.ac.in (N.K. Nagwani), sarsij@mnnit.ac.in (S. Tripathi).

https://doi.org/10.1016/j.dajour.2023.100362
Received 30 June 2023; Received in revised form 30 October 2023; Accepted 8 November 2023
Available online 10 November 2023
2772-6622/© 2023 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

clustering technique is simple to implement and a new technique was created that combined the spectral clustering with the weighted kernel K-means[8]. For finding the community pattern in complicated networks by combining the classic spectral approach with a novel non-parametric Bayesian model [9]. The Spectral Clustering on Ratios of Eigenvectors is a new approach that has been proposed recently [10] which is the ratio of the first leading eigenvector to the other leading eigenvectors were used as the basis for the grouping. Although many algorithms have been developed, none of them have shown to be effective for every network.

To learn the continuous feature representations of network nodes, Node2vec provides a robust algorithm framework [11]. It generates node embeddings that capture the structural properties of the network by utilizing a mixture of random walks and skip-gram models. This model is capable of learning a mapping of nodes to a low-dimensional space of features that optimizes the probability of maintaining neighbor nodes in networks. To learn the task-independent representations in complex networks efficiently, this model also introduces efficient novel approaches [12]. One such algorithm is Sim2vec, which has been proposed by Xie et al. [13] in 2018. Sim2vec aims to encode more comprehensive node similarities among different nodes in a network by incorporating node similarity metrics into the node2vec framework. This approach improves the accuracy of node embeddings and enables better downstream tasks such as link prediction and community detection. Another approach, a community-based variational autoencoder model for learning network embeddings has been proposed by Shi et al. [14]. This approach consists of a community detection module and a deep learning module that jointly optimizes the objective function to learn a low-dimensional representation of the network that captures both the community structure and the node attributes. Besides, a deep sparse filtering method has been proposed for detecting network communities. This approach uses an unsupervised deep learning algorithm to find an efficient representation of the network that captures the underlying community structure [15]. Another approach has been introduced, that evaluate the learning communities via learner behaviors. This embeddings can be used to identify changes in learning patterns and predict future behavior [16]. In addition, a novel community detection method, motif-based embedding label propagation algorithm, has been developed. This algorithm employs network motif mining to obtain higher-order topology, while lower-order connectivity attributes are used to reconstruct a complex network topology [17]. A novel technique for simultaneous network embedding and community discovery in directed networks has been proposed [18]. The network embedding approach uses two vectors to represent out and in-nodes separately, allowing the same nodes to belong to distinct communities.

The most adaptable approach is provided by deep learning because deep learning models: (a) learn nonlinear network features like the interaction between nodes, (b) give a low-dimensional network representation that preserves the intricate network structures, and (c) increase the effectiveness in finding communities from disparate pieces of input, deep learning models present the most adaptable solution. Thus, deep learning has recently been a popular area of study for community detection because of its usefulness in processing graph-structured data using graph neural networks methods [6,19]. Graph convolutional networks are a type of neural network designed to work with graph data structures [20]. In the context of community detection, the GCN can be used to capture the structural information of a graph, such as the connections between nodes, and use this information to identify communities within the graph. There are two main classes of community detection methods based on GCNs: Supervised/Semi-supervised community classification and unsupervised network representation. Graph neural networks have emerged as a flexible and powerful approach for node classification in graph-based datasets, as well as for community detection and other graph inference tasks. Similar to other models based on neural networks, the graph neural network model is dependent on the choice of hyper-parameters, and fine-tuning is an essential process for optimizing their performance. Based on the previous studies, a brief summary of the challenges that need to be addressed in this study.

- The extraction of continuous node features and structural similarity, which pertains to the potential linkages or roles among nodes while maintaining the underlying characteristics of communities.
- Integrating edge data and node similarity in network analysis poses an analytical challenge, as it requires proficiently handling noise and redundancy while leveraging these two distinct perspectives on network information.

To address these issues, many studies and surveys have been undertaken on graph neural network to determine the best ways to tune their hyper-parameters and to investigate recent developments in graph-based learning and its many possible applications. Additionally, some researchers have proposed novel methodologies for incorporating node categorization with the graph neural network to discover communities in diverse networks. In this paper, community detection based on Node2Vec via graph convolutional network (CD2NE-GCN) has been presented to discover different community partitions from the viewpoint of optimizing node embedding. The Node2Vec has been utilized specifically to construct the input embedding vectors associated with each node and to locate the community partitions using a graph convolutional network that depicts the strength of nodes affiliating to communities while minimizing the loss function. To further illustrate the community partition, transform the community affiliation weighted matrix into a community affiliation matrix. The main contributions are listed as follows:

- This work aims to provide a new approach to identifying communities using node embedding using Node2Vec via graph convolutional networks (CD2NE-GCN). The proposed methods combine the Node2Vec process and message passing through feed forward network (FFN) via graph convolutional networks (GCNs) with clustering approach.
- To acquire the accurate structural relationship of nodes and identify the node's community they belong through node embedding in a complex graph network.
- The proposed approach aims to combine node and structural similarities to create a unified similarity matrix, which is then transformed into a lower-dimensional space. The purpose of this integration is to enhance the accuracy and effectiveness of community detection algorithms.
- Five different real-world networks have been used in experiments, and the findings show that CD2NE-GCN consistently outperforms the state-of-the-art alternatives.

The remaining portions of the article are described here. Section 2 presents a detailed literature review on community detection and node embedding using graph neural network approaches. Section 3 provides an overview of the proposed model's structure and formulation, also the graph convolutional network approach has been discussed. In Section 4, The experimental settings and the discussion of baseline methodologies with proposed approaches is pointed out in this part. Experimental results and discussions are discussed in Section 5. Section 6, discusses the practical implication of community detection. At last, in Section 7, the conclusion and future work were addressed.

2. Related works

Different researchers have proposed a plethora of different approaches to finding communities. Optimizing measures that quantify the quality of community structure like Modularity is one path to uncovering community structure. Inferring the connection between vertices and communities using node embedding and graph neural

network is yet another approach to community detection. In this section, we mainly focus on node embeddings and graph neural network techniques. These methods can be broadly divided into methods based on certain node embedding approaches and graph neural networks like Graph Convolutional Network (GAN), Generative Adversarial Networks (GANs), Graph Attention Networks (GAT), and Autoencoder (AE) Model and methods based on Graph Representation.

2.1. Community detection using node embedding techniques

Node representation learning is a machine learning technique used in graph analysis to learn node embedding. The "embedding" means mapping each node in a network into a low-dimensional vector representation of nodes in a graph, which will give insight into nodes' similarity and network structure. Node embedding is a special kind of encoding method that represent node in a graph to vectors [21]. The main idea of Node embedding is to capture the structural and semantic properties of the nodes, such as their connections to other nodes and their semantic similarity with other nodes [22,23]. Perozzi et al. have proposed a novel approach called DeepWalk [24]. This technique extends the most cutting-edge research in language modeling and unsupervised feature learning over word sequences to graphs. It is a random walk-based approach that leverages the idea of word embeddings from natural language processing (NLP) to generate node embeddings. The basic idea behind DeepWalk is to treat a graph as a corpus of vertices, and to generate node embeddings by applying the skip-gram model from NLP.

2.2. Community detection using graph neural networks

Recently, Graph Neural Networks have emerged as a promising method for processing graph-structured data using deep learning algorithms, sparking new areas of study in the field. Community detection strategies based on Graph Attention Networks are effective even in highly complex network environments. For networks with multiple types of relations, the Graph Attention Networks aggregate neighborhood-level features of nodes using trainable weights that pay consideration to various factors [25]. Deep Graph Infomax [26] (DMGI) is a graph attention network for community detection, which embeds each relation type separately and then maximizes the globally shared features to find communities. Furthermore, Metapath Aggregated Graph Neural Network [27] (MAGNN) provides an improved method for detecting communities by using multi-informative semantic metapaths to differentiate between various graph attention layers. Moreover, Adversarial training is useful for generative models and enhances discrimination, but when applied to community detection, it must be combined with a solution to the over-fitting issue [28,29]. A generative adversarial network based approach like Seed Expansion with generative Adversarial Learning (SEAL) [30] creates seed-aware communities from specified seed nodes using Graph Pointer Network incremental updates (iGPN). Community-level components include generator, discriminator, seed selector, and location. The discriminator uses Graph Isomorphism networks (GINs) to label produced communities. The locator sends generator regularization signals to reduce extraneous nodes in community detection.

Furthermore, Autoencoders (AEs) including stacked, sparse, denoising, convolutional, and variational are employed in unsupervised community detection [20]. Autoencoders may describe nonlinear, noisy real-world networks and generate smooth ones. Autoencoders uses an encoder and decoder. The encoder converts a high-dimensional network topology and possible properties into a low-dimensional latent feature space. From encoder representations, the decoder builds a network. Wide-ranging applications, such as dynamic community detection, can easily make use of approaches based on stacked autoencoders. A stacked AE-based community detection algorithm Semi-supervised Nonlinear Reconstruction Algorithm with deep neural network (semi-DRN) [31] has been developed where the nonlinear node representations learned by the modularity matrix in autoencoders and the K-Means method have been used for getting community partitions. Integrating graph convolutional networks into autoencoders has been useful because GCNs give high-order graph regularization and Autoencoders fix the problem of GCNs being too smooth [32]. Another method has been proposed Structural Deep Clustering Network [33] (SDCN) in which author integrates the structural information into deep clustering and applied duel self supervised optimization. In other approach, One2Multi Graph Autoencoder for Multi-view Graph Clustering [34] (O2MAC) has been used for multi-view graph clustering.

Furthermore, A new community detection algorithm based on Markov Stability and Graph Neural Network (CDMG) [35] has been proposed. Markov Stability is used as the loss function to measure community partition efficacy and the community affiliation weight matrix quantifies node affiliation with communities. Optimizing Markov Stability using graph neural networks finds communities. Another novel encoding method has been developed that involves initially assigning a continuous variable to each edge inside a graph network. Subsequently, the concatenation of these continuous variables is transformed into a partition solution for the attribute network in question using a sequence of nonlinear functions [36]. Additionally, The present study introduces a novel approach for estimating community influence by employing a graph convolutional network within a semi-supervised learning framework [37,38]. This approach takes into account the interplay between qualities and the global graph information, hence facilitating the identification and construction of influential communities. In addition, using a non-negative matrix factorization, the proposed structurally equivalent embedding approach for community recognition translates similarities into a low dimensional vector space (SENMF) [39]. Moreover, The method suggested in this study, referred to as GAAE, incorporates an attention mechanism guided by community identification based on non-negative matrix factorization (NMF) to acquire node representations. Additionally, NMF is employed to simultaneously factorize these representations and reveal the underlying community structure [40,41]. Moreover, a deep learning model that has been proposed (VNSA) effectively combines both network organization and node properties by incorporating convolutional neural networks and community division indicators [42]. Another Graph attention auto-encoder and modularity maximization model has been proposed a community detection approach that utilizes unsupervised attributed network embedding (CDBNE) in order to leverage clustering-oriented information [43]. The literature review has been summarized in Table 1.

A systematic analysis of the literature reveals that the field of deep learning-based node classification and community discovery is still in its early stages. The findings provided in this section indicates that a methodology for integrating deep learning-based node classification into the process of identifying communities in intricate graph-based data is required. To solve the aforementioned gap, the current effort aims to provide researchers with a structured technique for adapting the methodology in order to obtain more accurate results. Key notations in the paper are described in Table 2.

3. Methodology of the proposed model

The nodes in an undirected network, denoted by $G = \{V, E\}$, are connected to one another via a set of edges, or links, denoted by $\eth E\eth$, and V is a set of nodes in a graph and denoted by $\eth V\eth = N$. The purpose of community detection is to group nodes together into K communities. An appropriate representation for this kind of assignment is a adjacency matrix A of community membership and it is represented as $A = \{A_{ij}\} \in R^{N\times K}$, where,

$$A = \begin{matrix} T \\ \end{matrix} \begin{matrix} 1; & \text{if there is an edge between } v_i \text{ and } v_j: \\ 0; & \text{otherwise:} \end{matrix} \quad (1)$$

Table 1
Highlights of the literature review.

Reference paper	Methods used	Purpose	Performance
Yuan et al. 2022 [35]	Graph Convolutional Network approach.	The proposed CDMG create node embeddings using a graph neural network and trained to provide a weight matrix showing the strength of nodes connecting with communities. The training technique uses Markov Stability to reduce loss.	The NMI of the CDMG model is 82.95%.
Sun et al. 2022 [36]	Graph neural network encoding method.	Initially associating each edge in a graph network with a continuous variable, then transforming the concatenation of the continuous variables to a partition by considering attribute network.	The accuracy of the model is 97%.
Hussein et al. 2022 [37]	Semi-supervised graph convolutional network approach.	The proposed model retrieving the community influence using a graph convolutional network in a semi-supervised learning setting.	The accuracy of the model is 80.87%.
Sobolevsky et al. 2022 [38]	Recurrent graph neural networks method.	The proposed technique uses unsupervised network community discovery via modularity optimization using recurrent graph neural networks.	The accuracy for proposed method is 96.56%.
Zhu et al. 2022 [39]	Non-negative matrix factorization approach.	Using a non-negative matrix factorization, the proposed structurally equivalent embedding approach for community recognition translates similarities into a low-dimensional vector space (SENMF).	The accuracy for SENMF model is 70.2%.
He et al. 2022 [40]	Non-negative matrix factorization with graph attention autoencoder approach	The suggested method (GAAE) uses an attention mechanism led by NMF community identification to obtain node representations and factorize them simultaneously.	The performance of GAAE method is better than other benchmark methods.
Berahmand et al. 2023 [41]	Non-negative Matrix Factorization Approach.	The proposed AGNMF-AN method uses augmented Attributed Graph (AAG) is applied to combine both the topological structure and attributed nodes and update the affinity matrix.	The accuracy of the model is 81.23%.
Wang et al. 2023 [42]	Variational graph autoencoders approach.	The deep learning model that has been proposed (VNSA) effectively combines both network organization and node properties by incorporating convolutional neural networks and community division indicators.	The modularity accuracy of the model is 44%.
Zhou et al. 2023 [43]	Graph attention auto-encoder and modularity maximization approach.	The model being proposed introduces a community detection approach that utilizes unsupervised attributed network embedding (CDBNE) in order to leverage clustering-oriented information.	The accuracy of the model is 72.8%.
Chaudhary et al. 2023 [44]	Graph Neural Network approach.	The Gumbel-SoftMax method is employed to extract the important characteristics necessary for revealing concealed communities within intricate networks.	Two factors, the degree matrix and the adjacency matrix, as well as loops, were incorporated to boost performance.

Table 2
Definitions of the significant symbols.

Symbols	Narration
G	A Graph
$G = (V; E; X)$	Graph-Based Data as input
Comm	Set of Communities
$\acute{A}$	Adjacency Matrix
D	Degree Matrix
w	Weighted Matrix
D^{f-1}	Diagonal Matrix
S_m	Scaling Strategy
L'	Normalized Graph Laplacian
F	Eigen Matrix

3.1. Node embedding approach for community detection

In graph-based data representation, node embedding techniques have attained extraordinary success in recent years. Because of its ability to learn a mapping of nodes to a low-dimensional space of features that increases the likelihood of preserving neighbor nodes in networks, the Node2vec [23] model has been selected to find communities in complex graph-structured data. The key contribution is the formulation of a variable notion of a node's neighborhood in a network [45]. The Node2vec can learn representations that arrange nodes according to their roles and functions in the network and the communities to which they belong if a suitable notion of a neighborhood is used. The following Eq. (2) is introduced by Node2Vec to calculate the probability of transitioning to node x from node y, with transition beginning at node u and the random walk of fixed length l has been simulated.

$$Prob(C_i = x \eth C_{i-1} = y) = \begin{cases} \frac{yx}{Z}; & \text{if } (y; x) \in E: \\ 0; & \text{otherwise:} \end{cases} \quad (2)$$

where C_i denote the ith node in the walk path; Un-normalized transition probability between nodes V_y and V_x is $_{yx}$ and Z is a normalized constant.

This strategy requires generating a set of biased random walks that can efficiently probe multiple regions surrounding a single node. This technique evolved a biased random walk algorithm that is both flexible and capable of exploring nearby nodes in a BFS or DFS-like fashion. The random walk can be understood by comparing it with walking. This technique starts at one node and jumps around the graph to the next node at random. In a stochastic manner, an adjacent node is selected and subsequently traversed. The aforementioned procedure

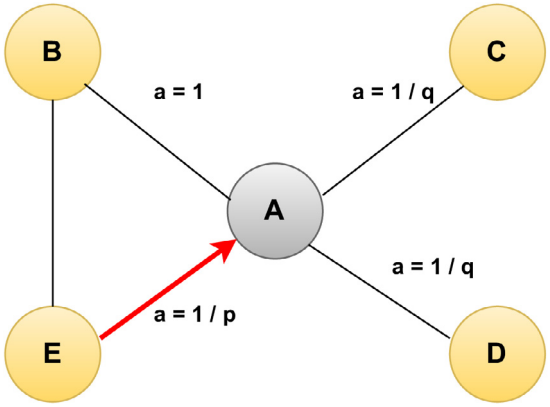

Fig. 1. Biased second order random walks.

is iteratively executed until a predetermined length of the walk is attained. The parameter known as walk length determines the length of the "sentences". The node2vec technique starts random walks from each graph node. Walks per node decide how many random walks start from a node. The node2vec algorithm generates phrases from graph nodes using random walks. The algorithm feeds random walk sentences into the skip-gram with a negative sampling (SGNS) model and retrieves the hidden layer weights as node embeddings [46].

However, Node2Vec uses biased random walks of the second order in its implementation. First-order random walks simply depend on their present state. The present and preceding states are considered in detail using second-order walks [23]. Node2Vec method determines traversal probability by taking into account its preceding step. In 2nd Order random walk with two parameters – the weight of width p and the weight of depth q – the Node2vec technique is able to uncover previously concealed relationships between nodes. The probability of making it from the starting node to the ending node in an unweighted, undirected network graph is calculated by following the edges between the nodes as illustrated in Eq. (3). By setting the un-normalized transition probabilities of edges based on $_{yx} = {}_{pq}(t; x):W_{yx}$,

$$ {}_{pq}(t; x) = \begin{cases} \frac{1}{p}; & \text{if } D_{tx} = 0: \\ 1; & \text{if } D_{tx} = 1: \\ \frac{1}{q}; & \text{if } D_{tx} = 2: \end{cases} \tag{3} $$

where D_{tx} represents the geodesis path between nodes V_t and V_x. The range of D_{tx} is [0, 1, 2], therefore only two parameters are needed for guided walks.

For instances, the walk just moved from node E to A and is considering its next move. The return parameter p controls the probability of retracing and instantly revisiting a node. Setting parameter p high reduces the likelihood of revisiting a node and eliminates 2-hop sampling redundancy. This method promotes moderate graph exploration. However, a low p number increases backtracking, keeping the random walk closer to the initial node as shown in Fig. 1.

3.2. Graph neural network approach

Further, graph neural networks have shown remarkable success in accurately expressing data in graphs. As a result, For recognizing clusters in intricate graph structures, previously settled on the graph convolution network (GCN). Graphs (the evolving structure of node connections) are an example of the non-Euclidean data type that GCNs may process. In contrast, convolutional neural networks (CNNs) work on uniformly structured data, such as images and text data. In other words, graph convolution is a way to adapt CNN to handle graph-based data instead of the more traditional Euclidean data [47,48]. Graph convolutional Networks (GCNs) are functionally equivalent to CNNs with regards to the 'convolution' operation. In short, this is a multiplication operation, with the weights being filters or kernels, applied to the input neurons. It is possible to calculate the "filter" function that operates as a signal on graph nodes by transforming both the filter and the signal into the Fourier domain, multiplying them, and then transforming them back into spectral form. Modifying signals by multiplying them by their graph Laplacian eigenvectors. Nonlinear activation functions in the context of neural networks are necessary for deep network training because they permit the learning of complicated connections in the underlying graph data without resorting to linear activation functions. The input activation of a node or an output can be converted into an average of weighted inputs using an activation function. Nonlinear functions like the rectified linear activation function (ReLU) have positive outputs when the input is straight and zero outputs otherwise. Numerous neural networks start with this activation function since it is simple to train and produces better results than others.

3.3. Design principles of the proposed model

Commonly utilized essential parameters are presented in Table 1 and the main technique of the suggested study (CD2NE-GCN) is illustrated in Fig. 2. The CD2NE-GCN is designed for recognizing communities in graph-based networks. The suggested approach consists of three stages; stage one takes the graph network G as input and pre-processes it. In the pre-processing step, the process starts with the input of a graph and the subsequent generation of random walks. Each node in this representation of the walks is a word, therefore the whole thing makes sense in terms of a directed sequence of words. Then, the skip-gram model receives the created random walks. Each node in the random walk can be interpreted as a word, and the complete walk can be interpreted as a sentence, making the skip-gram model applicable to both words and sentences. Each node is assigned an embedding based on the outcome of the skip-gram model. The skip-gram with negative sampling (SGNS) approach is used by the node2vec algorithm. Using the specified node as the origin, the node2vec algorithm produces a sequence of random walks. However, the node2vec technique uses biased random walks of the second order. Both the present and the previous state are considered by the Second-order walks. In other words, the algorithm takes into account its past state while determining traversal probabilities. Firstly, evaluating the weighted matrix W_{xy} by using Eq. (4). Every node is represented by an embedding, and the weight W_{xy} of an edge connecting nodes V_x and V_y is equal to the similarity S_{xy} of the two embeddings.

$$ W_{xy} = S_{xy} = \exp\left(-\frac{\tilde{n}V_x - V_y\tilde{n}_2^2}{2^2}\right) \tag{4} $$

where $\tilde{n}V_x - V_y\tilde{n}_2^2$ Euclidean Distance, weight matrix $W = \{W_{xy}$ð1 f x f n; 1 f y f n and n is number of nodes in a network. Secondly, calculate the degree matrix D by setting diagonal scores of W to 0 i.e. $W(x; x) = 0$ as mentioned in Eq. (5) and Laplacian matrix $L = D - W$.

$$ D_x = \sum_{y=1}^{\text{É}} W_{xy} \tag{5} $$

In third step, evaluate normalized Laplacian matrix through $L' = D^{-1/2}LD^{-1/2}$, followed by finding the eigenmatrix F. The first part of the proposed model (CD2NE-GCN) is presented from Line number 5–10 in Algorithm-1. In the second phase, generated node embeddings associated with each node have been supplied to the graph convolutional network as input. Further, obtained normalized the sum of all feature vectors that a node and its neighbors (including itself) ÁX where Á is denotes adjacent matrix and X represents feature vectors, that possess in order to produce a weighted average matrix of all the embedded

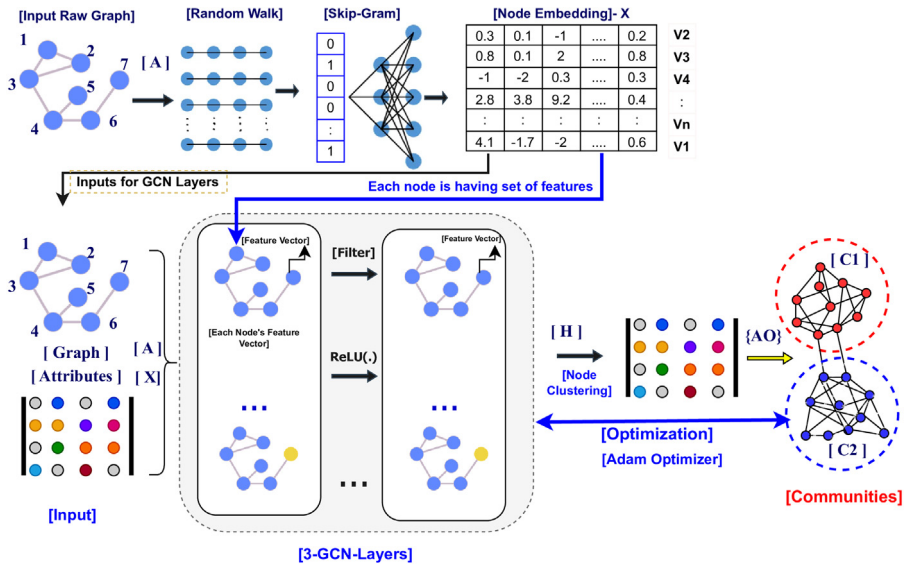

Fig. 2. Framework for node embedding GCN-based community detection.

vectors generated by the Node2vec method for low-dimensional nodes as shown in Eq. (6).

$$A = \hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2} \tag{6}$$

After obtaining the normalized term, the average weighted matrix was passed on to the 3-layer GCN with the ReLU and Softmax [49] activation function have been used as mentioned in Eq. (7).

$$Z = f(A;X) = \mathrm{softmax}(\ (\hat{D}^{-1}AH^{i}W^{i})) \tag{7}$$

where i is an integer that ranges between [0, 1, 2] and the weight matrix $W^{i} \in R^{(H \times F)}$ is the input to the hidden layer H. An activation function and weight parameter called Softmax and ReLU ( ) have been used to make predictions [49,50]. In the last phase, the final output is obtained by applying clustering approach to refined result from GCN's result. Using an activation function, the final layer is trained to produce a ranked list of communities. The loss function is easily calculated by comparing the estimated community labels to the partial ground-truth labels and using the larger of the two values as the loss.

The mechanism of the CD2NE-GCN model is described in Algorithm-1. In the final stage, the improved result from the GCN is submitted to the Adam optimizer to produce the final output via node clustering (Spectral clustering). The proposed model uses gradient calculations to update weight parameters and is trained for 300 epochs at an initial learning rate of 0.001. In this paper, a paradigm for node embedding GCN-based community detection has been presented. It takes as its starting point a graph structure (A) to be embedded into a vector space (V) through the Node2vec method. Graph latent characteristics are refined in several layers of a Graph Convolutional Network (GCN) to meet the specifications of community detection. The ReLU() function initiates the process of learning graph representations. We demonstrate a framework for identifying communities. Communities are identified using node clustering on embeddings (H), and the optimal community affiliations can be maximized using evaluations such as Mutual Information (MI). Using convolutional representations of node embeddings, the outcomes of clustering are optimized together with the GCN representations that eventually discover each node into communities. The suggested model is given training by the Adam optimizer [51] for a total of 300 epochs, with an initial learning rate of 0.001; this rate is used to calculate gradients, which are then used to update the weight parameters (i.e. $W^{(0)}; W^{(1)}$ and $W^{(2)}$). The activation function guarantees that the final layer's output is a ranked list of communities.

Algorithm 1 The Proposed Technique (CD2NE-GCN) Algorithm

Input: Graph Data set G

Data: Parameters : d,p,q,m |Vector dimension is d; the weight of breadth is p; the weight of depth is q; number of communities m.

Output: Predicting various communities $Comm = (C_1; C_2; ::: ; C_n)$

Load Graph Network Data set $G = (V; E; X)$ and begin random walks.
foreach $k \leftarrow 1$ *to* N **do**
  Determine the probability of edges without normalization .
end
Rand-Walks=0
foreach $i \leftarrow 1$ *to* R **do**
  foreach $j \leftarrow 1$ *to* N **do**
    Rand-Walks= node2vec(G)
  end
end
Compute w = getW // Generate the Weighted Matrix w.
Compute D = getD(w) // Generate the Degree Matrix D.
Compute L = D − W // Obtain Laplacian Matrix L.
Evaluate Normalized Laplacian Matrix L′.
Evaluate eigenmatrix F.
foreach $i \in N$ **do**
  $\hat{A}X$:Message Passing Mechanism generates a total of all neighbors' feature vectors, including itself.
  Calculate average feature vector of all neighbors by $S_m = \hat{D}^{-1}\hat{A}\hat{D}^{-1}X$
  Get normalized weighted matrix $A = \hat{D}^{-1/2}\hat{A}\hat{D}^{-1/2}$.
end
foreach $l \in L$ **do**
  Evaluate feature vectors Layer-wise to obtained node embedding.
  Obtained the final node embeddings of each node by using ReLU and Softmax function H.
end
Cl-F = SpectralClustering(m).
Comm = getLabels(Cl-F-fit(H)).
Optimize the results Comm by Adam Optimizer.

Table 3
Brief overview of the benchmark data sets.

Data sets	Vertices	Edges	(CC)	Communities	Attributes	Type of network
Karate	34	78	0.57	2	–	Social
Citeseer	3312	4715	0.14	6	3703	Citation
Cora	2708	5429	0.24	7	1433	Citation
Facebook	4031	88 234	0.6	193	1283	Friendship
DBLP	371,080	1,049,866	0.63	13,477	–	Bibliography

(CC) denotes average clustering coefficient, while the "Attributes" denotes range of attribute features.

4. Experiments

4.1. Datasets

The experiment conducted to demonstrate the effectiveness of CD2NE-GCN was performed on several different types of network datasets, including citation graph datasets (Citeseer, and Cora), social network datasets (Karate and Facebook). These datasets have been selected because they are publicly available benchmark datasets and have been used as evaluation datasets by other methods. The statistics of these datasets are summarized in Table 3 in the study. The selection of these datasets allows for a comprehensive evaluation of CD2NE-GCN on a range of network types, from citation networks to social networks, and provides insights into its performance in different scenarios. By comparing the results of CD2NE-GCN with those of the baseline methods on these datasets, the authors aim to show the superiority of their proposed method in learning node representations in complex networks.

- **Karate Club:** The Karate Club network is often used as a benchmark for evaluating community detection algorithms because it has a clear division of nodes into two well-defined communities. There are a total of 34 nodes in the network, which correspond to the club's membership. The network's 78 edges represent member relationships [52].
- **CiteSeer:** The CiteSeer dataset is a representation of scientific publications and their citation links using bag-of-word features. Each publication is described using a binary word vector, where a value of 1 indicates the presence of a specific word in the lexicon (which has a total of 3703 different terms) and a value of 0 indicates its absence. With 3312 scientific publications represented as nodes and 4732 citation links, the CiteSeer dataset provides valuable insights into the relationships between scientific papers and the knowledge they contribute to the field. This information can be useful for research in areas such as information retrieval, text classification, and bibliometrics [53].
- **Cora:** The 2708 scientific papers found in Cora have been organized into seven distinct subject areas. The total number of citation links is 5429. Each record in the dataset is given a 0/1 vector value to signify whether or not the specified lexical term appears in the record. There are 1433 unique words in the lexicon [53].
- **Facebook:** A friend network with the intention of detecting groups, including both network topology and user credentials, is represented by a dataset of friend links. The profiles and networks of 4039 Facebook users have been gathered from 193 circles across 10 ego networks [54].
- **DBLP:** The DBLP is a digital library of bibliographic references in computer science. Authors, papers, terms, and publication types have all been collected from the DBLP collection. There are four distinct research foci represented by the contributors. Each writer is represented by a "bag of words" made up of the papers they have written. Co-authorship networks can be found in fields as diverse as chemistry, computer science, medicine, and engineering [55].

4.2. Evaluation criteria details

The effectiveness of the community detection results has been evaluated using an evaluating metric index.

- **Accuracy (ACC):** Accuracy (ACC) is a frequently employed evaluation metric that quantifies the proportion of instances in a dataset that has been correctly categorized [20]. The calculation is as follows:

$$\text{Accuracy}(S_i; S'_j) = \frac{\sum_{i=1}^{\delta S_i \delta} (S_i; C'_j)}{\delta S \delta} \tag{8}$$

  Here, when the labels of the ground truth and the identified communities are the same, the Kronecker delta ((:)) equals 1.
- **Normalized Mutual Information (NMI):** NMI is commonly used to evaluate the effectiveness of a community detection algorithm, that employs the information theory criterion for comparing the discovered communities with the ground-truth communities. A higher value is preferable because its values vary from 0 to 1 [56].

$$NMI(S_i; S'_j) = \frac{-2 \sum_{i=1}^{S_i} \sum_{j=1}^{S'_j} N_{(ij)} \log \frac{N_{(ij)} \times n}{N_{(i)} : N_{(j)}}}{\sum_{i=1}^{S_i} N_{(i)} \log \frac{N_{(i)}}{n} + \sum_{j=1}^{S'_i} N_{(j)} \log \frac{N_{(j)}}{n}} \tag{9}$$

  where parameter N denotes the community matrix, N_i is the ith row of the community's matrix sum and N_j signifies the sum of values in the jth column of the community matrix. Further, parameter S_i is the value of normal community result and The outcome of the algorithm's community results is denoted by S'_j.
- **F1-Score:** It has been determined that the F1-Score [20] for community detection corresponds to the harmonic mean of Precision and Recall, which can be written as follows :

$$F1 - Score(S_i; S'_j) = 2 : \frac{\text{precision}(S_i; S'_j) : \text{recall}(S_i; C'_j)}{\text{precision}(S_i; S'_j) + \text{recall}(S_i; S'_j)} \tag{10}$$

  where S'_i stands for the specific community and S_i is a stand for the real-world community/group.
- **Modularity:** Modularity (Q) is the most well-known optimization functions. It provides an evaluation of the quality of community-based segregation of the network [57]. Modularity can be stated in its broadest sense as:

$$\text{Modularity}(Q) = \frac{1}{2m} \sum_{i;j=1}^{\text{E}} (A_{ij} - \frac{K_{ij}}{2m}) \ (S_i; S_j) \tag{11}$$

  The chance that two nodes belong to the same community is defined by the delta function $(S_i; S_j)$, where A is the adjacency matrix, m is the number of edges in the graph, K_i is the degree of node i. Also, the algorithm performs well when the input value is in the range $Q \in -1; 1$, where -1 indicates that no two nodes belong to the same community and 1 indicates that the two nodes belong to the same community [58].

4.3. Baselines methods

The proposed method CD2NE-GCN has been compared with different type of baseline methods that consist of Node Embedding

(Node2Vec), GCN based algorithms like Community Deep Graph Infomax (CommDGI), Adaptive Graph Encoder (AGE) and Graph Attention Network (GAT). Besides, Graph Adversarial Network techniques such as Community Detection with Generative Adversarial Nets (CommunityGAN), and autoencoder-based methods like Stacked Auto Encoder (SAE), GCN-based approach for Unsupervised Community Detection (GUCD) methods.

- **Node2Vec (N2V):** Node2Vec maps similar network neighborhoods to similar vector representations of graph nodes in a low-dimensional space. The algorithm builds these representations by randomly walking the graph and maximizing a similarity objective function. Node2Vec's random walk technique balances exploring graph regions with exploiting local neighbors [11].
- **Community Deep Graph Infomax (CommDGI):** The Community Mutual Information (MI) on nodes and communities is used by Deep Graph Infomax (CommDGI) to improve graph representations and clustering while simultaneously measuring network Modularity for maximum effect. It uses k-means clustering to find the most central nodes in each group [59].
- **Graph Attention Network (GAT):** Self-attention process for computing node embedding is the core concept behind the Graph Attention Network (GAT). In a network, self-attention is a method that allows for the dynamic allocation of weights to other nodes in the graph based on their relevance to the current node. In this way, the network may compute node embedding by paying attention just to the most relevant regions of the graph [25].
- **Graph Adversarial Networks (GAN):** GANs consist of generators and discriminators. The generator creates fake samples from random noise, and the discriminator tries to distinguish them. The discriminator learns to recognize samples from the generator. The generator and discriminator are taught adversarial to produce more realistic instances and distinguish real from fraudulent ones. A low-dimensional vector of random noise is transformed into a high-dimensional sample via the generator network. The discriminator network estimates the sample's authenticity [60].
- **Stacked Auto Encoder (SAE):** Unsupervised feature learning and dimensionality reduction are two common applications of stacked autoencoders (SAEs), a special kind of neural network. The autoencoder architecture, of which this is a variation, is used. As the name suggests, a stacked autoencoder stacks many layers of these encoder–decoder pairs, with the output of each layer feeding into the input of the next. Learning a hierarchical representation of the input data is the goal of stacked autoencoders. The autoencoder learns to extract more abstract features from the output of the previous layer, allowing it to capture ever more subtle correlations and patterns in the data. The autoencoder's final layer generates a compact representation of the input data that can serve as a feature vector in further processing steps [61].
- **GCN-based approach for Unsupervised Community Detection (GUCD):** The GCN-based approach for Unsupervised Community Detection (GUCD) learns a graph representation using a Graph Convolutional Network (GCN) and identifies community structure using spectral clustering. The GUCD involves graph embedding and community detection. The GCNs learn a low-dimensional graph representation during graph embedding. Local graph structure and node properties train the GCN to anticipate node embedding. Node embedding store graph topology for downstream clustering. To determine community structure, spectral clustering is applied to the learned node embedding. The spectral clustering divides nodes into communities by embedding similarity. The graph Laplacian matrix determines the number of communities automatically [62].

4.4. Setting of parameters

Two parameters, width weight p and depth weight q, are introduced by Node2Vec to define a second random walk. The probability of making it from the originating node to the destination node is calculated by following all of the edges and nodes in an unweighted and undirected network graph. The efficacy of the CD2NE-GCN method relies heavily on the choices made for its numerous parameters. After running multiple simulations, we found that $p = 1$, $q = 1$, $r = 10$, $d = 1$, and 10 walks in each network produced the best results for node embedding training. Each network's confirmed value of m, the number of communities, must match the actual number of groups present in the datasets description.

5. Results and discussions

The outcomes are presented in Table 4, and the assessment metrics (modularity, accuracy, NMI, and F1) are shown as percentages. Exciting progress has been made across all six datasets. This demonstrates that the suggested method is superior to the baseline methods on two out of three measures, providing strong evidence for its viability. For both attributed and unattributed graph data, the provided CD2NE-GCN model is preferable. With the exception of the NMI and F1-Score on the DBLP dataset, the CD2NE-GCN has outperformed the baseline approaches and obtained higher accuracy across the board. The GAT, Node2Vec, and GUCD approaches excel in comparison to other baseline techniques in terms of accuracy, NMI, modularity, and F1 Scores.

The proposed method CD2NE-GCN is able to give a more accurate and stable community structure than competing algorithms by achieving the highest accuracy, modularity for all the dataset, NMI score, and F1-score in most circumstances. When compared to other algorithms in Cora, Karate and Facebook produce the best results in terms of NMI and F1-Scores. The suggested technique outperforms its competitors in both Citeseer and DBLP; in particular, CD2NE-GCN's Accuracy and modularity value are significantly higher than those of competing algorithms. Because both node embedding with GCN deeply relies on network structure, CD2NE-GCN is responsive to the change of network structure, which is why CD2NE-GCN can identify community structure well. This may explain why CD2NE-GCN performed so favorably in these experimental networks.

5.1. Over-fitting and over-smoothness prevention

Message Passing captures non-Euclidean data properties in GCN models. This makes the development of a reliable system for forwarding messages both essential and difficult. Over-fitting and over-smoothness are common issues that develop after a model has been trained using deep GCN [63]. The problem of over-fitting arises when a model attempts to learn too much from the training data while still accounting for the noise present in the data. As a result, the model performs very poorly on training data or test data. Improve the performance of the suggested model by penalizing complex models and lowering loss and complexity with the activation function, which was incorporated to the loss function as a new term.

Also, over-smoothness is a problem [64,65]. Because of its inherent smoothness features, the deep GCN is especially vulnerable to over-smoothing. In particular, the effectiveness of subsequent tasks like node classification suffers when a large number of graph convolutional layers are stacked on top of one another, making the nodes in different classes (or clusters) indistinguishable and inseparable. Providing nodes to groups and independently normalizing them produces a new embedding matrix for the next layer. By adding this buffer, the Group Distance Ratio can be improved [65]. The ratio of two average distances of nodes is the result of using the Group Distance Ratio function. By doing so, nodes' representations are standardized when embedded within a group, while scaling with observable parameters yields diverse embeddings associated with different labeled groups. The suggested model uses a loss function to provide feedback to a trainable matrix assignment, which is then used to correctly label nodes.

Table 4
Analysis of CD2NE-GCN method efficiency compared to other baseline community detection methods across five graph networks (mean).

Datasets	Metric	Baselines methods						
		Node2Vec	CommDGI	GAT	GAN	SAE	GUCD	CD2NE-GCN
Karate	ACC	74.52	60.49	84.34	67.23	81.89	88.54*	**98.45**
	NMI	79.43	65.45	80.12	69.9	78.42	84.41*	**89.21**
	Modularity	66.78	73.19	78.43*	70.28	68.17	78.66*	**82.89**
	F1-Score	52.8	61.62	36.08	53.8	57.7	62.2*	**83.62**
Cora	ACC	87.67	75.43	89.67*	72.87	68.33	85.52	**99.04**
	NMI	70.58	66.12	79.56*	67.53	51.08	71.11	**83.51**
	Modularity	80.33	61.45	82.43*	72.39	63.87	82.67	**89.45**
	F1-Score	69.74	58.21	79.45*	64.74	53.63	66.45	**84.98**
Citeseer	ACC	86.94*	85.12	83.31	58.83	64.13	82.94	**96.39**
	NMI	80.64*	56.61	32.44	49.3	53.2	77.16	**85.32**
	Modularity	82.29*	73.19	75.02	76.32	68.17	78.66	**86.59**
	F1-Score	75.51	65.11	67.57	52.01	57.72	77.07*	**83.24**
Facebook	ACC	73.31	76.22	66.82	71.04	56.19	91.67*	**98.79**
	NMI	67.5	50.28	84.04*	49.69	55.51	82.55	**88.82**
	Modularity	66.78	73.19	75.02	78.43	68.17	84.26*	**90.89**
	F1-Score	64.25*	63.12	41.28	60.96	57.14	61.82	63.48
DBPL	ACC	58.16	60.31	92.65*	51.43	58.59	62.05	**97.67**
	NMI	53.34	31.17	71.45	53.49	68.21	80.59*	78.65
	Modularity	78.26	73.19	78.02*	75.43	68.17	68.66	**84.33**
	F1-Score	52.62	61.33	31.92	52.53	58.69	61.75*	**67.71**

* The top outcomes are highlighted in bold, whereas the most promising results from the baseline methods are denoted by an asterisk sign.

5.2. Examining the proposed model theoretically

The trials are carried out on five standard datasets, including karate, Citeseer, Cora, Facebook, and DBLP. Earlier work has extensively used these datasets, and most community detection methods are field-tested on them. The Node2Vec, CommDGI, GAT, GAN, Stacked Auto Encoder, and GUCD are the baseline algorithms chosen for the comparative analysis. The efficiency of the outlined approach is demonstrated and proven by the comparative analysis based on accuracy, NMI score, F1 Score, and Modularity. Table 2 provides an overall overview of the findings and percentage values for the assessment parameters (accuracy, NMI, F1, and Modularity). The five data sets yielded impressive results. These results show that the recommended technique is better than existing ones because the CD2NE-GCN approach consistently outperforms the baseline methods on nearly all of the metrics. The proposed CD2NE-GCN model works best with both attributed and unattributed graph data. The CD2NE-GCN has demonstrated superior accuracy to baseline methods on all datasets with the exception of DBLP. When compared to other algorithms, GUDA and GAT excel in DBLP, achieving the maximum possible ACC, NMI, Modularity, and F1 scores.

5.2.1. Feasibility of the proposed model

There have been several successful implementations of the proposed paradigm in fields as diverse as networks of friends, recommendations, computational biology, etc. By analyzing their online behavior, such as friend discovery, content perusal, and ad targeting, social network communities can be located and correlated. Community detection has other applications in protecting user privacy and apprehending internet offenders. Criminal ideology propagators and possible terrorists can be tracked down via community detection. Shoppers sharing comparable likes and consumption patterns may be readily discovered through community exploration, which contributes to the creation of efficient product suggestion systems, making this an important area of promotional and advertising research.

5.2.2. Comparative performance of various convolutional layers

The efficiency of the proposed frameworks model improved by a mean of 19.97% for 3-Convolutional Layers, as shown in Table 5. At this point, all trials utilizing a 2, 3, or 4-layer GCN model have concluded.

Table 5
Model performance can vary depending on the number of convolutional layers used in the various datasets.

Datasets	2-Layers	3-Layers	4-Layers
Karate	67.45	86.62	68.44
Cora	61.32	85.87	71.98
Citeseer	56.54	84.79	64.32
Facebook	69.49	87.12	72.62
DBLP	58.89	69.18	74.21

5.2.3. Computational cost of the proposed model

The recommended method combines an attributed and unattributed network with the benefits of node embeddings via Node2Vec and message passing via graph convolutional networks in order to discover fundamental community groupings utilizing attributed information. The 32 GB RAM and Intel Core 3.4 GhZ computer have been used for all tests. The PyTorch Python modules have been used to create deep learning models [66]. Gephi [67] has been used to generate features for nodes in graph networks.

The running times of each technique are presented in Table 5. The running time statistics showed the worst time of the GAN was 3.21 s The table data indicates that the proposed methodology, CD2NE-GCN, exhibits the shortest execution times of 0.98 and 1.03 s for the Citeseer and Facebook datasets, respectively. Meanwhile, the remaining methods maintain their respective positions. This outcome is visually presented in Fig. 3. The time taken for each iteration, measured in seconds and considering the variability of feature sizes among datasets, it is of interest to determine the timing, measured in seconds, required to complete each cycle. In terms of computational efficiency, GAT, GUCD, Node2vec, and CD2NE-GCN exhibit superior performance when compared to other benchmark approaches (see Table 6).

6. Practical implications of the proposed model

This work aims to identify and group nodes with high similarity values, helping researchers separate them into relevant communities. Many of the problems with conventional machine learning techniques have been fixed by migrating to graph neural network models. Embedded representations of nodes (or documents) are learned by the GCN model based on the properties of the node and its neighbors. The CD2NE-GCN method has many implications for different networks such

Table 6
Running time on the Facebook and Citeseer datasets.

Methods	Time (s) Facebook	Time (s) Citeseer
Node2Vec	1.86	1.16
CommDGI	2.11	2.36
GAT	1.39	1.28
GAN	3.21	2.98
StackAE	2.02	1.79
GUCD	1.28	1.14
CD2NE-GCN	**1.03**	**0.98**

Fig. 3. Computational time chart of various methods for community detection.

as biological networks, recommendation systems, social networking sites, and website networks, and related fields. Using this strategy, researchers and analysts may better identify and classify node characteristics. This study used node embeddings via graph neural networks to probe its substantial effect on performance in locating heterogeneous communities in various locations. A technical article may cite a scientific one, and vice versa, in the network of papers that have been cited. Similar fields of articles (nodes) are clustered together into their own communities using this method (CD2NE-GCN). Each publication's word-presence binary vectors serve as the node features. In addition, the proposed model aids in the identification of hacker communities and social network terrorist networks by identifying characteristics such as participation in criminally motivated behavior on social media, promotion of criminal ideologies, and commission of other terrorist offenses [68].

Furthermore, the CD2NE-GCN method can be used to categorize communities whose users share common tastes and interests in terms of product purchases and website exploration. Manufacturers can utilize these classified groups to find users who are interested in comparable items. Both researchers and practitioners can benefit from the proposed approach because it provides a framework for categorizing communities for the purpose of making more informed decisions across a range of domains.

7. Conclusion and future work

Different communities can emerge in networks due to shared functional and structural characteristics across nodes. Consequently, gaining knowledge about the various communities inside a network can provide insights into its underlying structure and the basic concepts that determine its functioning. This paper proposed a novel node embedding using Node2Vec via a graph convolutional networks-based community detection method called CD2NE-GCN, which classifies the nodes by learning the continuous feature representations of the nodes using the node2vec and Graph Convolutional Network approach. The performance of this model is notable in scenarios involving outliers and sparse graph structures due to the incorporation of weighted averaged feature information from shared neighbors in conjunction with the properties of the node itself. The proposed model exhibits superior performance compared to the baseline approaches across many evaluation metrics, including Accuracy, modularity, NMI, and F1 Score, with improvements ranging from 3.2% to 19.27%, 4.6% to 10.27%, 4.5% to 12.06%, and 1.51% to 9.16%, respectively, for detecting communities. Furthermore, the proposed model achieves these improvements with significantly less computational cost. As a result, the proposed method can only detect unique communities, demonstrating that each node genuinely belongs to a distinct category.

Future research work on the proposed model will focus on two main objectives. Firstly, it can explore the application of graph convolutional neural network-based node embedding, explicitly utilizing the node2vec method, to detect overlapping communities in complex networks. Secondly, it can investigate the utilization of embedding approaches for dynamic attributed networks, explicitly focusing on incorporating overlapping community detection.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] M.A. Javed, M.S. Younis, S. Latif, J. Qadir, A. Baig, Community detection in networks: A multidisciplinary review, J. Netw. Comput. Appl. 108 (2018) 87–111.
[2] Z. Chen, X. Li, J. Bruna, Supervised community detection with line graph neural networks, 2017, arXiv preprint arXiv:1705.08415.
[3] G. Salha-Galvan, J.F. Lutzeyer, G. Dasoulas, R. Hennequin, M. Vazirgiannis, Modularity-aware graph autoencoders for joint community detection and link prediction, Neural Netw. 153 (2022) 474–495.
[4] X. Wang, P. Cui, J. Wang, J. Pei, W. Zhu, S. Yang, Community preserving network embedding, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 31, 2017, pp. 203–209.
[5] F. Hu, J. Liu, L. Li, J. Liang, Community detection in complex networks using Node2vec with spectral clustering, Physica A 545 (2020) 123633.
[6] M. Fiedler, Algebraic connectivity of graphs, Czechoslovak Math. J. 23 (2) (1973) 298–305.
[7] G.H. Golub, C.F. Van Loan, Matrix Computations, JHU Press, 2013.
[8] I.S. Dhillon, Y. Guan, B. Kulis, A Unified View of Kernel K-Means, Spectral Clustering and Graph Cuts, Citeseer, 2004.
[9] F. Hu, Y. Zhu, J. Liu, Y. Jia, Computing communities in complex networks using the Dirichlet processing Gaussian mixture model with spectral clustering, Phys. Lett. A 383 (9) (2019) 813–824.
[10] J. Jin, Fast Community Detection by Score, Project Euclid, 2015.
[11] A. Grover, J. Leskovec, node2vec: Scalable feature learning for networks, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 855–864.
[12] P. Goyal, E. Ferrara, Graph embedding techniques, applications, and performance: A survey, Knowl.-Based Syst. 151 (2018) 78–94.
[13] Y. Xie, M. Gong, S. Wang, W. Liu, B. Yu, Sim2vec: Node similarity preserving network embedding, Inform. Sci. 495 (2019) 37–51.
[14] W. Shi, L. Huang, C.-D. Wang, J.-H. Li, Y. Tang, C. Fu, Network embedding via community based variational autoencoder, IEEE Access 7 (2019) 25323–25333.
[15] Y. Xie, M. Gong, S. Wang, B. Yu, Community discovery in networks with deep sparse filtering, Pattern Recognit. 81 (2018) 50–59.
[16] C. Li, L. Zhao, Q. Zeng, H. Duan, Z. Zhao, Learner2vec-based learner community evolution analysis–a case study involving student card data, IEEE Access 7 (2019) 27416–27425.
[17] C. Li, Y. Tang, Z. Tang, J. Cao, Y. Zhang, Motif-based embedding label propagation algorithm for community detection, Int. J. Intell. Syst. 37 (3) (2022) 1880–1902.
[18] J. Zhang, X. He, J. Wang, Directed community detection with network embedding, J. Amer. Statist. Assoc. 117 (540) (2022) 1809–1819.
[19] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, M. Sun, Graph neural networks: A review of methods and applications, AI open 1 (2020) 57–81.
[20] X. Su, S. Xue, F. Liu, J. Wu, J. Yang, C. Zhou, W. Hu, C. Paris, S. Nepal, D. Jin, et al., A comprehensive survey on community detection with deep learning, IEEE Trans. Neural Netw. Learn. Syst. (2022).